

Parallel Computing: Who Steals Your Speedup

Lab course • Lecture 0

Where we are going today

Theory promises an N -fold speedup. Four thieves take it back: the algorithm, the hardware, the OS, and your own locks.



Every act names one thief, shows you exactly what it steals — and hands you the counter-move. We finish by putting all four on one checklist.

Concurrency is not parallelism

“Concurrency is about *dealing with* lots of things at once. Parallelism is about *doing* lots of things at once.”
— Rob Pike, 2012

Concurrency

- Structure the program as independent tasks
- Many tasks are in flight, but they take turns
- You want responsiveness and overlapped I/O
- Works on one core: the OS hands out time slices

? QUESTION FOR YOU

One thread on one core downloads three files at once. Concurrency or parallelism?

Parallelism

- Run independent work at the same time
- One goal only: finish sooner (speedup)
- Needs real hardware — separate execution units
- Lives inside one core too: SIMD, superscalar issue

Concurrency is not parallelism

“Concurrency is about *dealing with* lots of things at once. Parallelism is about *doing* lots of things at once.”
— Rob Pike, 2012

Concurrency

- Structure the program as independent tasks
- Many tasks are in flight, but they take turns
- You want responsiveness and overlapped I/O
- Works on one core: the OS hands out time slices

? QUESTION FOR YOU

One thread on one core downloads three files at once. Concurrency or parallelism?

Parallelism

- Run independent work at the same time
- One goal only: finish sooner (speedup)
- Needs real hardware — separate execution units
- Lives inside one core too: SIMD, superscalar issue

→ ANSWER

Concurrency. The requests just sit on the network and the core waits on I/O — nothing is computed at the same time.

counter++ is really three instructions

One line in your source, three separate steps for the CPU.

? QUESTION FOR YOU

Two threads run counter++ a million times each. What number gets printed?

C++

```
int counter = 0; // shared variable
void work() {
    for (int i = 0; i < 1000000; ++i)
        counter++;
}
std::thread a(work), b(work);
a.join(); b.join();
std::cout << counter << "\n"; // 2000000?
```

Java

```
static int counter = 0;
static void work() {
    for (int i = 0; i < 1000000; i++)
        counter++;
}
Thread a = new Thread(App::work), b = new Thread(App::work);
a.start(); b.start(); a.join(); b.join();
System.out.println(counter); // 2000000?
```

counter++ is really three instructions

One line in your source, three separate steps for the CPU.

? QUESTION FOR YOU

Two threads run counter++ a million times each. What number gets printed?

C++

```
int counter = 0;           // shared variable
void work() {
    for (int i = 0; i < 1000000; ++i)
        counter++;
}
std::thread a(work), b(work);
a.join(); b.join();
std::cout << counter << "\n"; // 2000000?
```

One increment, as GCC expands it on x86-64

```
mov  eax, DWORD PTR counter[rip] ; 1. LOAD
add  eax, 1                      ; 2. ADD
mov  DWORD PTR counter[rip], eax ; 3. STORE
```

The thread can be descheduled between any two steps, and a second core can cut in right in the middle.

Java

```
static int counter = 0;
static void work() {
    for (int i = 0; i < 1000000; i++)
        counter++;
}
Thread a = new Thread(App::work), b = new Thread(App::work);
a.start(); b.start(); a.join(); b.join();
System.out.println(counter); // 2000000?
```

→ ANSWER

Anything from 2 to 2 000 000. On my laptop it printed **1 187 431** — the increments overwrite each other, and the number is different every run.

Where the lost increment goes

One bad interleaving and two increments add up to +1 instead of +2.

Step	Thread A	Thread B	counter in memory
1	load eax = 0	—	0
2	—	load eax = 0	0
3	add eax = 1	—	0
4	—	add eax = 1	0
5	store counter = 1	—	1
6	—	store counter = 1	1

Both increments ran, yet memory holds 1: step 6 blindly overwrote step 5.

! DON'T DO THIS

Never assume `counter++` is atomic. It is a *data race*: undefined behavior in C++, and in Java a silent stream of lost updates.

Task, data, pipeline: three shapes of parallel work

Almost everything you parallelize is one of these three shapes, or a mix of them.

1. Task parallelism

different jobs, different code

Task A (CPU 0)

Task B (CPU 1)

- Each thread runs different code
- They may share the same data
- Balance the jobs, or one thread lags

Example: a web server

2. Data parallelism

one operation, many data chunks

D_0 | D_1 | D_2 | D_3

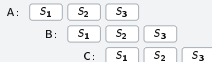
↓ the same $f(x)$

- Every thread takes its own chunk
- All of them run the same operation
- Equal chunks and cache locality set the speed

Example: adding two arrays

3. Pipeline parallelism

stages run at the same time



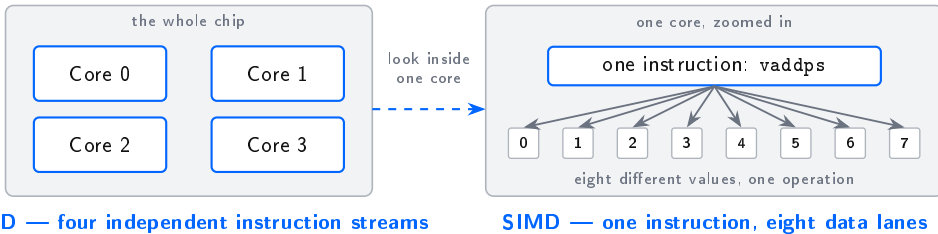
- One item walks through every stage
- Different items sit in different stages
- The slowest stage sets the pace

Example: a compiler

Real programs mix all three — and each shape invites its own thief: **task** loses to imbalance, **data** to the memory system, **pipeline** to its slowest stage.

One chip, two kinds of parallelism

Asked how much parallelism a chip offers, almost everyone answers with the core count. That is one axis of two.



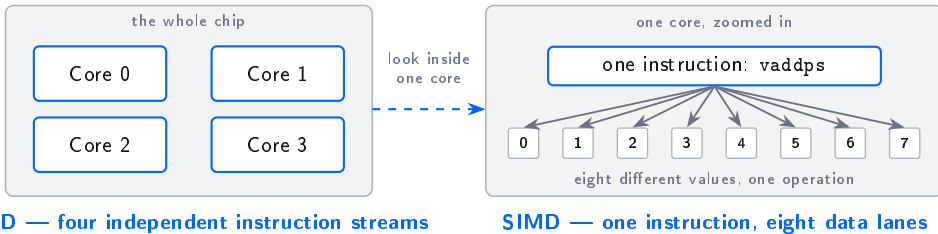
Flynn named four classes like this back in 1966. The fourth one, MISD, never really happened outside textbooks.

? QUESTION FOR YOU

Four cores, eight lanes each. How many float additions can this chip retire at once — and how many does your loop actually use?

One chip, two kinds of parallelism

Asked how much parallelism a chip offers, almost everyone answers with the core count. That is one axis of two.



Flynn named four classes like this back in 1966. The fourth one, MISD, never really happened outside textbooks.

? QUESTION FOR YOU

Four cores, eight lanes each. How many float additions can this chip retire at once — and how many does your loop actually use?

→ ANSWER

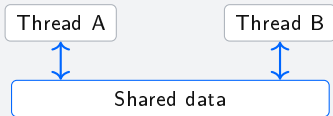
Thirty-two. A plain scalar loop on one thread uses **one** of them — and leaves 97% of the chip idle before a single lock is taken.

Shared memory or messages: how the pieces talk

The work is split. Now the parts have to exchange data — and you get to pick how.

Shared memory

everyone touches the same bytes



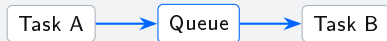
Threads read and write the same data.

Fast, but every concurrent write must be coordinated.

Example: a shared counter.

Message passing

nothing is shared, data is sent



Tasks send and receive messages.

The queue hands ownership over, so two tasks never share a payload.

Example: a job queue.

■ RULE OF THUMB

If handing off a task costs more than doing it, your tasks are too small — batch them.

Amdahl's law: the serial part sets the ceiling

First thief: the algorithm. Parse a file, then crunch the records independently.

One core: 100 seconds

5 s: parsing the file, record by record.

In this program that part is strictly serial.

95 s: crunching the records.

We split the records evenly across cores.

Start with two cores

Parsing: still 5 s. Crunching: $95/2$ s.

$$T(2) = 5 + 95/2 = 52.5 \text{ s}$$

$$\text{Speedup: } 100/52.5 \approx 1.90\times.$$

$$\text{For } N \text{ cores: } T(N) = 5 + \frac{95}{N} \text{ s, } S(N) = \frac{T(1)}{T(N)} = \frac{100}{5 + 95/N}.$$

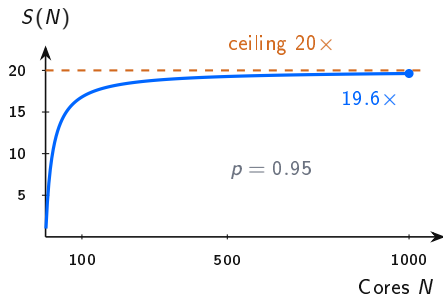
Same file every time, and we assume a perfect split with zero overhead.

? QUESTION FOR YOU

How much faster does this job run on 1000 cores? Could it ever finish in under 5 seconds?

Amdahl's law: the serial part sets the ceiling

First thief: the algorithm. Parse a file, then crunch the records independently.



→ ANSWER

$$T(1000) = 5 + 95/1000 = 5.095 \text{ s.}$$

$$S(1000) = 100/5.095 \approx 19.6\times.$$

You can never beat 5 s: the parsing never goes away.

The general form: Amdahl's law

$p = 0.95$: the share of one-core time that is parallel.

$$T(N) = T(1)((1 - p) + p/N)$$

$$S(N) = \frac{1}{(1 - p) + p/N} \xrightarrow{N \rightarrow \infty} \frac{1}{1 - p} = 20$$

? QUESTION FOR YOU

How much faster does this job run on 1000 cores? Could it ever finish in under 5 seconds?

Gustafson: what if the problem grows with the cores?

Second scaling law: hold the clock fixed and grow the problem instead.

Amdahl = strong scaling

The problem size is fixed and you want it sooner: a render, a build.

You can never pass the ceiling $1/(1 - p)$.

Gustafson = weak scaling

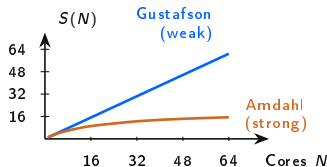
The time budget is fixed and you solve a bigger problem: a finer mesh, 4K instead of HD.

$$S(N) = (1 - p) + p \cdot N$$

With $p = 0.95$: $S(64) = 60.9\times$.

■ RULE OF THUMB

Ask first: am I holding the problem fixed, or the clock?



? QUESTION FOR YOU

Cores doubled, clock unmoved, twice the data crunched. Is that a win?

Gustafson: what if the problem grows with the cores?

Second scaling law: hold the clock fixed and grow the problem instead.

Amdahl = strong scaling

The problem size is fixed and you want it sooner: a render, a build.

You can never pass the ceiling $1/(1 - p)$.

Gustafson = weak scaling

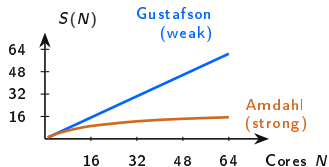
The time budget is fixed and you solve a bigger problem: a finer mesh, 4K instead of HD.

$$S(N) = (1 - p) + p \cdot N$$

With $p = 0.95$: $S(64) = 60.9\times$.

■ RULE OF THUMB

Ask first: am I holding the problem fixed, or the clock?



? QUESTION FOR YOU

Cores doubled, clock unmoved, twice the data crunched. Is that a win?

→ ANSWER

Yes — weak scaling. The serial part stayed fixed; only the parallel work grew.

Karp–Flatt: measure your own serial fraction

Amdahl needs p . Here is how you measure it on a program you already have.

? QUESTION FOR YOU

You measured $4.2\times$ on 8 threads. Good result or bad result?

Karp–Flatt: measure your own serial fraction

Amdahl needs p . Here is how you measure it on a program you already have.

? QUESTION FOR YOU

You measured $4.2\times$ on 8 threads. Good result or bad result?

The serial fraction you measure:

$$e = \frac{1/S(N) - 1/N}{1 - 1/N}$$

All you need is measured $S(N)$; $e \approx (1 - p)$.

■ RULE OF THUMB

One e tells you nothing. Plot e as N grows.

e flat $\Rightarrow$ serial code is the wall. Go parallelize it.

e grows $\Rightarrow$ overhead is piling up (the next three thieves).

Your measurements, and e from them:

N	$S(N)$	e
1	1.00	—
2	1.89	0.058
4	3.25	0.077 $\uparrow$
8	4.20	0.129 $\uparrow$
16	5.10	0.142 $\uparrow$

At $N = 8$: $e = (1/4.2 - 1/8)/(1 - 1/8) = 0.129$.

→ ANSWER

The $4.2\times$ alone tells you nothing. e climbs 0.058 $\rightarrow$ 0.142: overhead is growing. Profile before blaming the locks.

What happens after the plateau

Amdahl promises a plateau. Real machines do something else.

? QUESTION FOR YOU

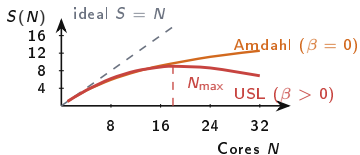
You keep adding threads past the plateau. Sketch the curve: where does it go?

What happens after the plateau

Amdahl promises a plateau. Real machines do something else.

? QUESTION FOR YOU

You keep adding threads past the plateau. Sketch the curve: where does it go?



$$S(N) = \frac{N}{1 + \alpha(N-1) + \beta N(N-1)}$$

α is waiting in queues, β is coherence chatter. Set $\beta = 0$ and you are back to Amdahl.

How to get them: measure $S(N)$ up to 32 cores and fit by least squares. Quick check: a peak at $N_{\max}$ means $\beta \approx 1/N_{\max}^2$.

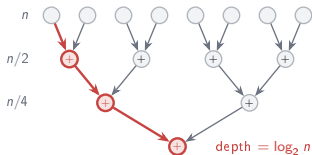
High $\alpha \Rightarrow$ something is serialized: shorten it, or split the shared resource. High $\beta \Rightarrow$ stop sharing mutable state.

→ ANSWER

It can. The penalty $\beta N(N-1)$ grows quadratically and drags the whole curve down.

Work and span: the algorithm's own ceiling

Work is every operation; span is the longest chain that waits on itself. Cores remove work, never the chain.



$T_1 = n$ (work), $T_\infty = \log_2 n$ (span, the red path). Parallelism $= n / \log_2 n$.
At $n = 10^6$: $T_\infty \approx 20$, so parallelism $\approx 50\,000$ — the point where extra cores stop helping, not a speedup. On 8 cores the algorithm is not what holds you back.

Two quantities, two floors:

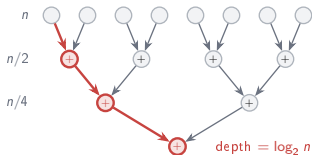
T_1 (work) — every operation, counted on one core. T_∞ (span) — the dependency chain you still walk on infinitely many cores.
Work law: $T_N \geq T_1/N$. Span law: $T_N \geq T_\infty$ — no hardware breaks that floor.

? QUESTION FOR YOU

So what is the span of a plain loop that does `sum += a[i]`?

Work and span: the algorithm's own ceiling

Work is every operation; span is the longest chain that waits on itself. Cores remove work, never the chain.



$T_1 = n$ (work), $T_\infty = \log_2 n$ (span, the red path). Parallelism $= n / \log_2 n$.
At $n = 10^6$: $T_\infty \approx 20$, so parallelism $\approx 50\,000$ — the point where extra cores stop helping, not a speedup. On 8 cores the algorithm is not what holds you back.

Two quantities, two floors:

T_1 (work) — every operation, counted on one core. T_∞ (span) — the dependency chain you still walk on infinitely many cores.
Work law: $T_N \geq T_1/N$. Span law: $T_N \geq T_\infty$ — no hardware breaks that floor.

? QUESTION FOR YOU

So what is the span of a plain loop that does `sum += a[i]`?

→ ANSWER

$T_\infty = n$: every step waits for the one before it, so the parallelism is exactly 1.

Roofline: are you limited by math or by memory?

Two ceilings: how fast you can compute, and how fast memory can feed you. The work you do per byte decides which one you hit.

Limit = $\min(\text{peak FP}, \text{BW} \times I)$, where I is FLOPs per byte moved from DRAM.

Vector add: $I = 1/12 \approx 0.08$

```
for (int i = 0; i < n; ++i) a[i] = b[i] + c[i];  
// 1 FLOP; bytes: b 4 + c 4 + a 4 = 12
```

Hopelessly memory-bound: the cores sit idle waiting for data.

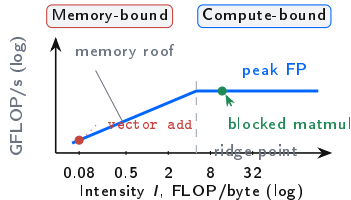
Blocked matrices, $B \times B$: I grows with B

```
// one B x B block: "2*B^3 FLOP per "3*B^2*4 bytes  
// I ~ B/6: B=8 -> 1.3, B=64 -> 10.7
```

Same algorithm, different traversal order — and now it is compute-bound.

■ RULE OF THUMB

Find the ridge point first. Left of it, raise I ; right of it, chase FLOPs.



? QUESTION FOR YOU

You sit left of the ridge point. Will doubling your threads help?

Roofline: are you limited by math or by memory?

Two ceilings: how fast you can compute, and how fast memory can feed you. The work you do per byte decides which one you hit.

Limit = $\min(\text{peak FP}, \text{BW} \times I)$, where I is FLOPs per byte moved from DRAM.

Vector add: $I = 1/12 \approx 0.08$

```
for (int i = 0; i < n; ++i) a[i] = b[i] + c[i];  
// 1 FLOP; bytes: b 4 + c 4 + a 4 = 12
```

Hopelessly memory-bound: the cores sit idle waiting for data.

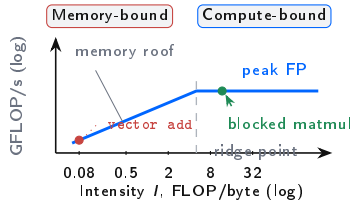
Blocked matrices, $B \times B$: I grows with B

```
// one B x B block: "2*B^3 FLOP per "3*B^2*4 bytes  
// I ~ B/6: B=8 -> 1.3, B=64 -> 10.7
```

Same algorithm, different traversal order — and now it is compute-bound.

■ RULE OF THUMB

Find the ridge point first. Left of it, raise I ; right of it, chase FLOPs.



? QUESTION FOR YOU

You sit left of the ridge point. Will doubling your threads help?

→ ANSWER

No — the memory bus is the wall. Raise I first, then add threads.

Memory hierarchy: what a cache miss really costs

Second thief: the hardware. It takes nothing out of your algorithm — it just makes it wait.

? QUESTION FOR YOU

A core needs one number that is not in any cache. How long does it wait?

Memory hierarchy: what a cache miss really costs

Second thief: the hardware. It takes nothing out of your algorithm — it just makes it wait.

? QUESTION FOR YOU

A core needs one number that is not in any cache. How long does it wait?

→ ANSWER

Roughly **200 cycles**. Independent work keeps flowing — but everything that **depends on that value** stops dead until it lands.

Measured latencies (7-cpu.com), 3 GHz

Level	Skylake	Zen3	Total, cycles
L1d (32 KB)	4 cycles	4 cycles	≈ 4
L2 (256 / 512 KB)	12 cycles	12 cycles	≈ 12
L3 (8 / 16 MB)	42 cycles	47 cycles	≈ 45
RAM (DDR4)	42 + 51 ns	47 + 74 ns	190 / 270

Last row = the full cost of a miss: DRAM plus the 42 cycles already spent missing in L1–L3. So “about 200” is 190 on this i7-6700, 270 on this Ryzen 5600G.

What this means in code

Walk row by row and the whole line pays off: 64 B is 8 numbers at once. Walk column by column and every element costs a new line and a new miss — same algorithm, several times the runtime.

One caveat: the famous “Latency numbers every programmer should know” (L1 3 cycles, mutex 50, RAM 100 ns, cross-datacenter 500 μs) is a teaching aid. Learn the ratios from it, never the absolute numbers.

Cache lines and MESI: reads are cheap, writes are not

A core cannot own one byte. The smallest thing the hardware moves is a whole 64-byte line.

? QUESTION FOR YOU

Two cores read the same line happily. One now **stores** to it — what happens to the other copy?

Cache lines and MESI: reads are cheap, writes are not

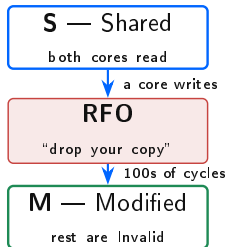
A core cannot own one byte. The smallest thing the hardware moves is a whole 64-byte line.

? QUESTION FOR YOU

Two cores read the same line happily. One now **stores** to it — what happens to the other copy?

→ ANSWER

It dies. Reads are free: both copies sit in **S (Shared)**. The write costs an **RFO** — claim the line, **invalidate every other copy**, then modify.



Why one byte costs a whole line

The core must own the entire 64-byte line before it touches one byte: broadcast, wait for every copy to drop, then write.

Two consequences, and both bite

- False sharing (next slide) — you pay in full even for different variables;
- Atomics and locks (Act IV) — a shared counter on 16 cores is slower than on one.

MOESI (AMD64) adds O (Owned): a dirty line goes core-to-core, skipping DRAM.

False sharing: one cache line, two victims

The worst bug in this act: the code looks perfectly parallel and runs like it is serial.

? QUESTION FOR YOU

Two threads write to **different** variables. Nothing is shared, no locks anywhere. Can they still cripple each other?

False sharing: one cache line, two victims

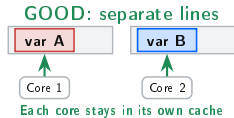
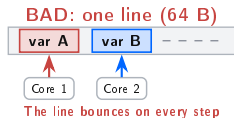
The worst bug in this act: the code looks perfectly parallel and runs like it is serial.

? QUESTION FOR YOU

Two threads write to **different** variables. Nothing is shared, no locks anywhere. Can they still cripple each other?

→ ANSWER

They can — if both land in the **same 64-byte line**. The hardware sees lines, not variables, so every write rips the line away from the other core.



How far it drops

- Drepper (4×P4): +390/734/1147 % on 2/3/4 threads — worse with every thread.
- Adding threads makes it slower, not faster.

How to fix it and how to find it

- C++: `alignas(64)`. Java: `@Contended` (JDK 8).
 - Find it: `perf c2c` names the hot line.
- Trap: hand-made padding gets optimized away.

Reordering: why “0 0” is a legal output

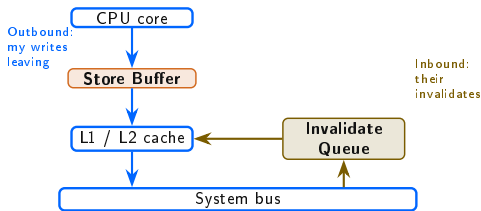
Your CPU does not run your program in order, and neither does your compiler.

Initially $x = y = 0$

```
// thread 1      // thread 2
x = 1;           y = 1;
print(y);        print(x);
```

? QUESTION FOR YOU

Try to get “0 0” by interleaving these four lines. Whichever order you pick, one store always lands first — so is “0 0” impossible?



- Your write waits in the store buffer, so your neighbor still sees the old value.
- Their invalidate waits in the invalidate queue, so you still read your stale copy.
- A fence does not “flush” these queues — it *constrains the order*: nothing after it may become visible before what came ahead of it.
- This is the theft: reordering is how the hardware buys speed, and every fence you add hands part of it back.

Reordering: why “0 0” is a legal output

Your CPU does not run your program in order, and neither does your compiler.

Initially $x = y = 0$

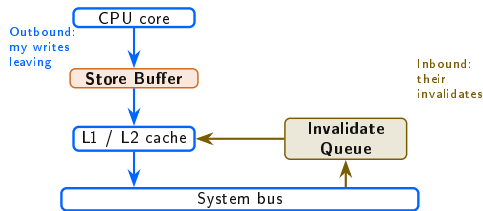
```
// thread 1      // thread 2
x = 1;           y = 1;
print(y);        print(x);
```

? QUESTION FOR YOU

Try to get “0 0” by interleaving these four lines. Whichever order you pick, one store always lands first — so is “0 0” impossible?

→ ANSWER

It happens on real x86 (**StoreLoad reordering**). Each core parks its own store in a store buffer and races ahead to the load — so both loads can run **before either store is visible** to the other core.



- Your write waits in the store buffer, so your neighbor still sees the old value.
- Their invalidate waits in the invalidate queue, so you still read your stale copy.
- A fence does not “flush” these queues — it *constrains the order*: nothing after it may become visible before what came ahead of it.
- This is the theft: reordering is how the hardware buys speed, and every fence you add hands part of it back.

Eight cores, and still not eight times

No locks, no sharing, perfectly independent threads — and the curve still flattens.

? QUESTION FOR YOU

Eight threads each stream through their own huge array. At how many threads does the speedup stop growing — and what decides that number?

Eight cores, and still not eight times

No locks, no sharing, perfectly independent threads — and the curve still flattens.

? QUESTION FOR YOU

Eight threads each stream through their own huge array. At how many threads does the speedup stop growing — and what decides that number?

→ ANSWER

Nothing. Saturated channels mean extra threads just queue for the same bytes. Measure **where** your curve flattens.

1. Bandwidth — the real ceiling

All cores share a **fixed number of channels**. Saturate them and more threads add queuing, not throughput. Measure with **STREAM** (McCalpin) and find the knee of your curve.

2. SMT hides stalls only

+5 % of die area buys +15–30 % when one thread stalls and the other has work. But both share the ALUs and L1: if the core waits for **memory**, the second waits too — SMT adds no bandwidth. In HPC it often loses. Measure both

3. NUMA splits the road in two

A multi-socket server has more than one set of channels, and the **other** socket's memory is slower:

```
numactl --hardware  
numactl --cpunodebind=0 ./app
```

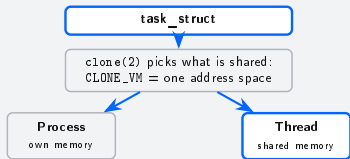
■ RULE OF THUMB

The hardware taxes you **per line** and **per byte**. Next thief: the OS, which taxes you **per thread**.

On Linux, a thread and a process are the same object

Third thief: the OS. Both are one `task_struct`; `clone(2)` decides what they share.

One structure, two modes



What to take away

- Threads share memory **by default** — that is where races come from.
- Processes are isolated, but pay for **expensive IPC**.
- A process is really a **thread group**; every thread in it costs real kernel resources.

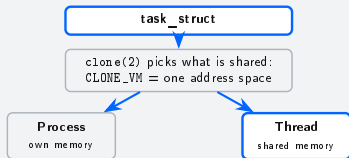
? QUESTION FOR YOU

Why does Chrome isolate sites in separate **processes** instead of one shared address space?

On Linux, a thread and a process are the same object

Third thief: the OS. Both are one `task_struct`; `clone(2)` decides what they share.

One structure, two modes



What to take away

- Threads share memory **by default** — that is where races come from.
- Processes are isolated, but pay for **expensive IPC**.
- A process is really a **thread group**; every thread in it costs real kernel resources.

? QUESTION FOR YOU

Why does Chrome isolate sites in separate **processes** instead of one shared address space?

→ ANSWER

Isolation. A crash in one site, or an exploit inside it, must not reach the rest of the browser. Chrome pays for that with expensive IPC, and it pays on purpose.

A million threads: why the OS says no

? QUESTION FOR YOU

`ulimit -s` gives every thread an 8 MB stack. How many threads fit into 16 GB of RAM?

A million threads: why the OS says no

? QUESTION FOR YOU

`ulimit -s` gives every thread an 8 MB stack. How many threads fit into 16 GB of RAM?

→ ANSWER

Not 2000 — far more. Those 8 MB are only a virtual reservation; pages are handed out when you actually touch them. You hit the kernel and the scheduler long before you run out of RAM.

Virtual is not physical

- 8 MB (`ulimit -s`) is just an **address range**, not memory you own.
- Physical memory (**RSS**) arrives page by page — only where you really wrote. Usually tens of KB per thread.

Where the real wall is

- Context switches, L1-L3 misses and TLB flushes.
- The scheduler: run queues, load balancing, migrations between cores.
- Hard OS limits: `RLIMIT_NPROC`, `threads-max`, `pid_max`.

What a thread costs: pthread, goroutine, Erlang, virtual

The lighter the thread, the more of them a runtime can carry — and the less work the OS kernel has to do.

pthread (OS)

Stack: 8 MB reserved by default; you can lower it only to `PTHREAD_STACK_MIN = 16 KB`.

A kernel object; the kernel picks who runs next. In practice: thousands.

Goroutine (Go)

Stack: starts at 2 KB and grows or shrinks on demand.

Scheduled in user space. Hundreds of thousands per process.

Erlang process

Stack+heap: 338 words at birth, about 2.7 KB.

Each one owns its heap. Default ceiling: 1 048 576.

Java virtual thread

The stack lives on the heap as a stack chunk, sized as needed.

Millions of tasks; the JVM schedules them in user space.

■ RULE OF THUMB

Pick the thread that matches the job: kernel threads for cores, user-space threads for waiting.

What virtual threads actually buy you

A million of them fit where a few thousand platform threads did. The interesting question is what that gets you.

When to reach for them

- **Yes:** thousands of tasks that spend their life **waiting on I/O**.
- **No:** CPU-bound work — you still stop at the number of physical cores.
- 10 000 tasks doing “sleep 1s”: a pool of 200 platform threads finishes ~200 per second, virtual threads ~10 000.

Traps to watch for

- **Pinning:** on JDK 21–23, blocking inside synchronized nailed the virtual thread to its carrier. JEP 491 fixed that in **JDK 24**.
- “*Virtual threads should never be pooled*” — pool the scarce resource, never the thread.
- A `ThreadLocal` holding something heavy will flood the heap.

? QUESTION FOR YOU

Your workload is pure number crunching on 8 cores. You move it to a million virtual threads. What changes?

What virtual threads actually buy you

A million of them fit where a few thousand platform threads did. The interesting question is what that gets you.

When to reach for them

- **Yes:** thousands of tasks that spend their life **waiting on I/O**.
- **No:** CPU-bound work — you still stop at the number of physical cores.
- 10 000 tasks doing “sleep 1s”: a pool of 200 platform threads finishes ~200 per second, virtual threads ~10 000.

Traps to watch for

- **Pinning:** on JDK 21–23, blocking inside `synchronized` nailed the virtual thread to its carrier. JEP 491 fixed that in **JDK 24**.
- “*Virtual threads should never be pooled*” — pool the scarce resource, never the thread.
- A `ThreadLocal` holding something heavy will flood the heap.

? QUESTION FOR YOU

Your workload is pure number crunching on 8 cores. You move it to a million virtual threads. What changes?

→ ANSWER

No. They scale waiting, not computing. Nothing about them adds cores to your machine.

The scheduler: who takes the CPU away from whom

A cooperative scheduler waits for the task to be polite. A preemptive one just takes the CPU back.

Go: the price of being polite

Before Go 1.14 a goroutine yielded only at a function call. So **a tight loop with no calls froze the whole runtime**.

Go 1.14 (2020) made preemption arrive as an OS signal: *“loops without function calls no longer deadlock the scheduler”*.

Linux always preempts; only the policy changed — EEVDF replaced CFS in kernel 6.6.

What a context switch costs

Case	Latency
OS thread, same core	1.2–1.5 μ s
OS thread, moved to another core	~ 2.2 μ s
Goroutine (user space)	~ 170 ns

The real bill is hidden: L1–L3 and the TLB go cold, and re-warming them costs more than the switch.

■ RULE OF THUMB

Oversubscribe — far more threads than cores — and the CPU burns its time shuffling them. Start from one rule: a CPU-bound pool $\approx$ the core count.

Priority inversion: guess who wins

Three threads with three different priorities, and exactly one shared mutex between them.

? QUESTION FOR YOU

The low-priority thread holds the mutex, the high-priority one waits for it. A medium thread that never touches the mutex is also runnable. Who gets the CPU?

Priority inversion: guess who wins

Three threads with three different priorities, and exactly one shared mutex between them.

? QUESTION FOR YOU

The low-priority thread holds the mutex, the high-priority one waits for it. A medium thread that never touches the mutex is also runnable. Who gets the CPU?

→ ANSWER

The medium one — the thread that has nothing to do with the lock. As long as it stays runnable it keeps preempting the lock holder, so the lock is never released and the high-priority thread misses its deadline.

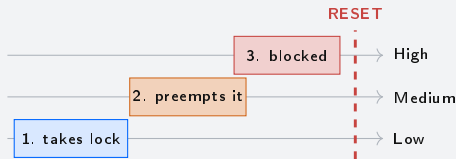
This hits your code, not just spacecraft

- **Thread pools:** a background task holds the lock a web request needs, while every worker is busy elsewhere. GC and service threads steal CPU from the lock holder too.
- **nice:** lower the priority of whoever owns a shared resource and you have just stalled the entire service.
- **The cure:** priority inheritance (PTHREAD_PRIO_INHERIT), fewer shared locks, shorter critical sections.

The same bug on Mars: Pathfinder, 1997

The lander touched down, started driving — and then began rebooting itself, over and over, from exactly the scenario we just walked through.

The 1553 bus, 8 Hz cycle



Low task locks the bus → medium task preempts it → high task blocks → **the watchdog resets the lander.**

Diagnosis, patch, lesson

- Logging saved them: a ring trace buffer let the lab on Earth reproduce the bug in 18 hours.
- The fix was uploaded to Mars: turn **priority inheritance** back on in `selectLib()`. Wind River had shipped it off *for speed*.
- “When you fly COTS stuff — make sure you know how it works” (G. Reeves).

So how many threads should you create?

The question you will actually type into a search box: how big is the pool?

CPU-bound: computing, not waiting

Use roughly as many threads as you have available cores.

Extra threads do not create extra cores. All they create is cache pressure and context switches.

In a container you may own fewer cores than the OS reports — check the cgroup limits.

I/O-bound: mostly waiting

A first estimate, not a law:

$$N \approx C \cdot \left(1 + \frac{W}{S}\right)$$

C is cores, W is time spent waiting, S is time spent computing.

8 cores and 90% waiting $\rightarrow N \approx 80$.

Thousands of such tasks? Switch to virtual threads.

■ RULE OF THUMB

The formula only gives you a starting point. Find the real pool size by profiling under load, never by guessing.

Data race $\neq$ race condition

Fourth thief: your own synchronization. It starts with two bugs everyone mixes up — and each one needs a different fix.

Data race — a bug in memory

One location, two threads, at least one write, zero synchronization.

```
int counter = 0;
void worker() {
    for (int i=0; i<1000000; i++)
        counter++; // data race
}
```

C/C++: plain undefined behavior.

Java: not UB, but you can read values out of thin air.

Race condition — a bug in logic

The outcome depends on who got there first. Happens without a data race too.

```
// map = ConcurrentHashMap
if (!map.containsKey(k)) // check
    map.put(k, v);        // then act
// do this instead: map.putIfAbsent(k, v);
```

Each call is atomic on its own. The pair of them is not.

? QUESTION FOR YOU

We make every field atomic. Do the races go away?

Data race $\neq$ race condition

Fourth thief: your own synchronization. It starts with two bugs everyone mixes up — and each one needs a different fix.

Data race — a bug in memory

One location, two threads, at least one write, zero synchronization.

```
int counter = 0;
void worker() {
    for (int i=0; i<1000000; i++)
        counter++; // data race
}
```

C/C++: plain undefined behavior.

Java: not UB, but you can read values out of thin air.

Race condition — a bug in logic

The outcome depends on who got there first. Happens without a data race too.

```
// map = ConcurrentHashMap
if (!map.containsKey(k)) // check
    map.put(k, v);        // then act
// do this instead: map.putIfAbsent(k, v);
```

Each call is atomic on its own. The pair of them is not.

? QUESTION FOR YOU

We make every field atomic. Do the races go away?

→ ANSWER

The **data race** does. The **race condition** stays: atomic steps never add up to an atomic sequence.

Atoms and CAS: synchronization on the cheap

An atomic never enters the kernel. But “atomic” does not mean “one instruction”: a CAS loop retries until it wins.

Ready-made atomic operations

```
std::atomic<int> counter{0};  
counter.fetch_add(1); // lock xadd
```

```
AtomicInteger c = new AtomicInteger();  
c.incrementAndGet();  
LongAdder a = new LongAdder(); // sharded
```

The price: the cache line goes Modified (RFO). Pennies on one core, painful on many.

CAS — the foundation of lock-free

```
int old = v.load();  
while (!v.compare_exchange_weak(  
    old, f(old)))  
    ; // old was refreshed, try again
```

“Write the new value, but only if the old one is still there.”
Missed it? Loop and retry.

ABA: the cell went $A \rightarrow B \rightarrow A$ behind your back and CAS sees nothing. Fix: carry a version counter.

? QUESTION FOR YOU

Eight threads hammer one counter. atomic or mutex?

Atomics and CAS: synchronization on the cheap

An atomic never enters the kernel. But “atomic” does not mean “one instruction”: a CAS loop retries until it wins.

Ready-made atomic operations

```
std::atomic<int> counter{0};  
counter.fetch_add(1); // lock xadd
```

```
AtomicInteger c = new AtomicInteger();  
c.incrementAndGet();  
LongAdder a = new LongAdder(); // sharded
```

The price: the cache line goes Modified (RFO). Pennies on one core, painful on many.

CAS — the foundation of lock-free

```
int old = v.load();  
while (!v.compare_exchange_weak(  
    old, f(old)))  
    ; // old was refreshed, try again
```

“Write the new value, but only if the old one is still there.”
Missed it? Loop and retry.

ABA: the cell went $A \rightarrow B \rightarrow A$ behind your back and CAS sees nothing. Fix: carry a version counter.

? QUESTION FOR YOU

Eight threads hammer one counter. atomic or mutex?

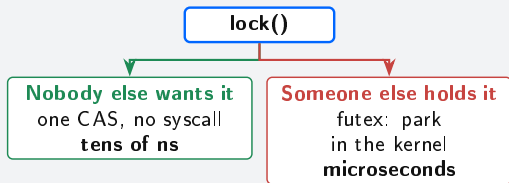
→ ANSWER

It depends on the fight. Light contention: atomic, easily. Eight cores hammering *one* cache line: the atomic is no better than a mutex — now you **shard** it (LongAdder).

What a mutex really costs

Everything depends on one thing: is anybody else holding it? The gap is a hundredfold.

One `lock()`, two very different stories



That is ≈ 20 ns against microseconds. You never pay for *the lock* — you pay for a thread that goes to sleep.

So why not write a spinlock?

It sounds right: the section is tiny, so parking the thread in the kernel must cost more.

Lock holder preemption: the OS scheduler evicts the thread that holds the lock. Everyone else keeps **burning CPU** while it sleeps.

On Linux the slow path is a **futex** (Fast User space muTEX). Before sleeping it re-checks the word atomically — otherwise a thread could fall asleep just after someone else's `unlock`.

■ RULE OF THUMB

In user space, take the plain mutex. Its fast path is already one atomic instruction — you will not hand-write anything cheaper, and the contended case is the kernel's job, not yours.

How coarse should your lock be?

One lock around the whole map is simple and safe — and it turns 16 cores back into one.

Coarse: one lock for everything

```
synchronized V get(K k) {...}  
synchronized void put(K k, V v) {...}
```

Everyone queues up: only **one thread at a time** is inside the map.

Fine: shard the lock

```
Lock b = lock[hash(k) % N]; // per bucket  
b.lock();  
try { ... } finally { b.unlock(); }
```

What splitting locks costs you

- You now grab several locks at once → **deadlock** risk
- `size()` and iteration stop being exact
- More memory, and much trickier code

In practice: don't roll your own — reach for `ConcurrentHashMap`.

? QUESTION FOR YOU

The service is crawling. The profiler says 80 % of the time is spent waiting on one mutex. What do you do first?

How coarse should your lock be?

One lock around the whole map is simple and safe — and it turns 16 cores back into one.

Coarse: one lock for everything

```
synchronized V get(K k) {...}  
synchronized void put(K k, V v) {...}
```

Everyone queues up: only **one thread at a time** is inside the map.

Fine: shard the lock

```
Lock b = lock[hash(k) % N]; // per bucket  
b.lock();  
try { ... } finally { b.unlock(); }
```

What splitting locks costs you

- You now grab several locks at once → **deadlock** risk
- `size()` and iteration stop being exact
- More memory, and much trickier code

In practice: don't roll your own — reach for `ConcurrentHashMap`.

? QUESTION FOR YOU

The service is crawling. The profiler says 80 % of the time is spent waiting on one mutex. What do you do first?

→ ANSWER

Don't start with sharding. First **shrink the critical section**: push I/O, allocation and logging out of it. That alone often fixes it.

Deadlock needs all four Coffman conditions

A deadlock happens only when **all four** conditions hold at once — so break any one of them.

The classic two-mutex trap

```
A: m1.lock();   B: m2.lock();  
A: m2.lock();   // waits for B  
B: m1.lock();   // waits for A
```

The four Coffman conditions:

1. **Mutual exclusion** — while one thread holds it, nobody else can.
2. **Hold and wait** — I keep one lock while asking for another.
3. **No preemption** — nobody can take the lock away.
4. **Circular wait** — the wait-for graph closes a loop.

The fix: kill the circular wait

1. **Always lock in the same order.** Every thread takes the locks in one fixed order (say, by memory address) — then no loop can form.

```
std::scoped_lock lk(m1, m2); // C++17
```

2. **Break hold and wait:** use `try_lock_for()`. If it fails, drop every lock you hold, back off a random amount, retry — otherwise you just trade deadlock for livelock.

? QUESTION FOR YOU

Thread A takes m1 then m2; thread B takes them the other way round. Does the program always hang?

Deadlock needs all four Coffman conditions

A deadlock happens only when **all four** conditions hold at once — so break any one of them.

The classic two-mutex trap

```
A: m1.lock();   B: m2.lock();  
A: m2.lock();   // waits for B  
B: m1.lock();   // waits for A
```

The four Coffman conditions:

1. **Mutual exclusion** — while one thread holds it, nobody else can.
2. **Hold and wait** — I keep one lock while asking for another.
3. **No preemption** — nobody can take the lock away.
4. **Circular wait** — the wait-for graph closes a loop.

The fix: kill the circular wait

1. **Always lock in the same order.** Every thread takes the locks in one fixed order (say, by memory address) — then no loop can form.

```
std::scoped_lock lk(m1, m2); // C++17
```

2. **Break hold and wait:** use `try_lock_for()`. If it fails, drop every lock you hold, back off a random amount, retry — otherwise you just trade deadlock for livelock.

? QUESTION FOR YOU

Thread A takes m1 then m2; thread B takes them the other way round. Does the program always hang?

→ ANSWER

No — and that's the **worst** part: your local tests stay green, and then it freezes solid under production load.

Happens-before: the one memory-model rule you need

The compiler and the CPU reshuffle your instructions freely. The memory model is the contract that says when you are *guaranteed* to see another thread's write.

The one rule to keep

If *A happens-before B*, then every write made before *A* is guaranteed visible at *B*.

Where you get an edge for free:

- `unlock(m) → the next lock(m);`
- **Java:** write to a `volatile` → read of it;
- **C++:** release-store → acquire-load *of the same atomic*;
- `start()` → first line of the new thread;
- thread finishes → `join()` returns.

One keyword, two different meanings

```
volatile boolean ready; // Java: OK
```

```
volatile bool ready; // C/C++: NOT OK
```

```
std::atomic<bool> ready; // use this
```

Java: real barriers, real happens-before.

C/C++: stops only the *compiler*. No CPU barriers, no happens-before — fine for hardware registers, useless between threads.

? QUESTION FOR YOU

A plain `bool stop`. One thread spins in `while (!stop)`, another writes `stop = true`. Does the loop ever exit?

Happens-before: the one memory-model rule you need

The compiler and the CPU reshuffle your instructions freely. The memory model is the contract that says when you are *guaranteed* to see another thread's write.

The one rule to keep

If *A happens-before B*, then every write made before *A* is guaranteed visible at *B*.

Where you get an edge for free:

- `unlock(m) → the next lock(m);`
- **Java:** write to a `volatile` → read of it;
- **C++:** release-store → acquire-load of *the same* atomic;
- `start()` → first line of the new thread;
- thread finishes → `join()` returns.

One keyword, two different meanings

```
volatile boolean ready; // Java: OK
```

```
volatile bool ready; // C/C++: NOT OK
```

```
std::atomic<bool> ready; // use this
```

Java: real barriers, real happens-before.

C/C++: stops only the *compiler*. No CPU barriers, no happens-before — fine for hardware registers, useless between threads.

? QUESTION FOR YOU

A plain `bool stop`. One thread spins in `while (!stop)`, another writes `stop = true`. Does the loop ever exit?

→ ANSWER

In C/C++ that is a data race, so the program has **undefined behavior** — and in practice the compiler hoists the read into a register and your loop becomes `while (true)`. Use `std::atomic<bool>`.

Condition variables: how to wait without burning CPU

A mutex protects data. A condition variable solves a different problem: how to *wait for something to happen* without spinning.

The producer–consumer pattern

```
// consumer
std::unique_lock lk(m);
while (q.empty())    // while, never if
    cv.wait(lk);    // releases m, then sleeps
auto item = q.front(); q.pop();
// producer
{ std::lock_guard g(m); q.push(x); }
cv.notify_one();    // wake up after unlock
```

What wait() actually does

1. Take the mutex.
2. Check the condition in a loop.
3. wait() *atomically* drops the mutex and sleeps, so no signal can slip past you.
4. You wake up **holding the mutex again**.
5. The loop re-checks the condition.

Why a loop: wakeups can be spurious, or another thread may have grabbed the item first.

? QUESTION FOR YOU

Swap that while for a plain if. What breaks?

Condition variables: how to wait without burning CPU

A mutex protects data. A condition variable solves a different problem: how to *wait for something to happen* without spinning.

The producer–consumer pattern

```
// consumer
std::unique_lock lk(m);
while (q.empty())    // while, never if
    cv.wait(lk);     // releases m, then sleeps
auto item = q.front(); q.pop();
// producer
{ std::lock_guard g(m); q.push(x); }
cv.notify_one();     // wake up after unlock
```

What wait() actually does

1. Take the mutex.
2. Check the condition in a loop.
3. wait() *atomically* drops the mutex and sleeps, so no signal can slip past you.
4. You wake up **holding the mutex again**.
5. The loop re-checks the condition.

Why a loop: wakeups can be spurious, or another thread may have grabbed the item first.

? QUESTION FOR YOU

Swap that while for a plain if. What breaks?

→ ANSWER

A spurious wakeup pops an empty queue — a crash you only ever see under load.

Lost wakeup: signal or broadcast?

A signal sent when nobody is waiting simply evaporates: a condvar has no memory and counts nothing.

The signal nobody heard



Takeaway: keep the state (`q.empty()`) in data under the lock. Data remembers; the condvar doesn't.

signal vs broadcast

- `signal()` wakes **exactly one** thread; `broadcast()` wakes them all.
- **The signal trap:** if threads wait on *different* conditions through one condvar, you may wake the wrong one. It sleeps again, the right one never wakes. **Everything hangs.**
- **The broadcast cost:** the whole crowd wakes and fights for one lock (*thundering herd*).

■ RULE OF THUMB

signal when *one* waiter can make progress. broadcast when several can, or when you cannot tell which one — and if the waiters want different things, give each condition its own condvar.

Fair locks: no starvation, less throughput

A strict queue saves you from starvation, and it costs you throughput.

Why an unfair lock runs faster

A thread that is **already running on a core** grabs the free lock on the spot, instead of waiting for a sleeper to be woken. That's *barging*.

A fair lock must hand the lock to whoever is first in line, and waking that thread takes microseconds.

The price of *barging*: a queued thread can be skipped over again and again. That is *starvation*.

How real runtimes handle it

- **Go** `sync.Mutex` is adaptive: barging by default, but a thread that has waited too long flips the lock into *starvation mode*, where it is handed over strictly in order.
- **Java** `ReentrantLock(true)` is the fair mode. The Javadoc says it outright: “*lower overall throughput, often much slower*”.

? QUESTION FOR YOU

You have a p99 latency SLA. Fair lock or unfair?

Fair locks: no starvation, less throughput

A strict queue saves you from starvation, and it costs you throughput.

Why an unfair lock runs faster

A thread that is **already running on a core** grabs the free lock on the spot, instead of waiting for a sleeper to be woken. That's *barging*.

A fair lock must hand the lock to whoever is first in line, and waking that thread takes microseconds.

The price of *barging*: a queued thread can be skipped over again and again. That is *starvation*.

How real runtimes handle it

- **Go** `sync.Mutex` is adaptive: barging by default, but a thread that has waited too long flips the lock into *starvation mode*, where it is handed over strictly in order.
- **Java** `ReentrantLock(true)` is the fair mode. The Javadoc says it outright: “*lower overall throughput, often much slower*”.

? QUESTION FOR YOU

You have a p99 latency SLA. Fair lock or unfair?

→ ANSWER

Still start unfair — it is faster on average. Switch to fair only when the **tail** breaks your SLA: watch p99/p999, never the average, because barging hides starvation there.

Mostly readers: RW-lock → seqlock → RCU

When reads vastly outnumber writes, a plain mutex wastes time — readers never disturb each other.

1. RW-lock

Many readers **or** one writer.

Catch: often *slower* than a mutex — every reader still writes to a shared atomic counter.

Also: writer starvation, and upgrading read→write deadlocks.

2. Seqlock

A version counter: the writer bumps it by +1 on entry and +1 on exit.

The reader **writes nothing**: it compares the version before and after, and re-reads if they differ.

Limit: anything the reader follows must outlive the read — pointers need a reclamation scheme.

3. RCU

Readers take **no lock** at all: they just read the old version.

The writer pays instead: it edits a copy, swaps the pointer atomically, and can only free the old memory once every reader has let go (the *grace period*).

Where it lives: the Linux kernel.

■ RULE OF THUMB

Don't reach for an RW-lock by reflex. It wins only when reads are **long** and frequent; for short critical sections a plain mutex almost always beats it.

Races are caught by tools, not by reading

A race shows up in one run and hides in the next. “I ran it 100 times and it was fine” proves nothing.

C/C++

ThreadSanitizer instruments every memory access and reconstructs happens-before:

```
g++ -fsanitize=thread -g -O1 a.cpp
```

It reports the race **even when that run came out correct** — but only among the accesses that actually ran. A clean run is evidence, not a proof. Costs $\approx 5\text{--}15\times$ slowdown.

Helgrind also flags risky mutex ordering.

JVM

jcstress, from the JDK team, runs a micro-test millions of times and **collects every outcome it actually observes**, flagging the ones the memory model forbids.

Lincheck (JetBrains) hammers your data structure with concurrent operations and, on failure, hands you a **reproducible interleaving** that breaks it.

>_ LIVE DEMO

```
g++ -fsanitize=thread -O1 race.cpp && ./a.out — the program prints a “correct” answer, and TSan immediately prints WARNING: data race with both stacks: who read and who wrote.
```

Which primitive do I actually need?

Read this table left to right: name the guarantee first, then take the simplest tool that gives it.

What you need	Primitive	What to watch out for
A counter, a flag, one value	atomic / CAS	Cheap; shard it under contention (LongAdder)
An invariant across several fields	mutex	Fast path needs no syscall; keep it short
Wait until something becomes true	condition variable	Always while; keep the condition in the data
Wait for N tasks to finish	latch / join	One-shot: you cannot rearm it
Sync threads phase by phase	barrier	The number of participants is fixed up front
Cap how many run at once	semaphore	No owner: any thread may release a permit
Many long reads, few writes	RW-lock / RCU / copy	Often slower than a mutex — measure
A ready-made thread-safe collection	concurrent containers	One call is atomic; a sequence is not

? QUESTION FOR YOU

Nothing in this table is your first move. What is?

Which primitive do I actually need?

Read this table left to right: name the guarantee first, then take the simplest tool that gives it.

What you need	Primitive	What to watch out for
A counter, a flag, one value	<code>atomic</code> / <code>CAS</code>	Cheap; shard it under contention (<code>LongAdder</code>)
An invariant across several fields	<code>mutex</code>	Fast path needs no syscall; keep it short
Wait until something becomes true	<code>condition variable</code>	Always <code>while</code> ; keep the condition in the data
Wait for <i>N</i> tasks to finish	<code>latch</code> / <code>join</code>	One-shot: you cannot rearm it
Sync threads phase by phase	<code>barrier</code>	The number of participants is fixed up front
Cap how many run at once	<code>semaphore</code>	No owner: any thread may release a permit
Many long reads, few writes	<code>RW-lock</code> / <code>RCU</code> / <code>copy</code>	Often slower than a <code>mutex</code> — measure
A ready-made thread-safe collection	<code>concurrent containers</code>	One call is <code>atomic</code> ; a sequence is not

? QUESTION FOR YOU

Nothing in this table is your first move. What is?

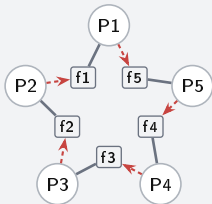
→ ANSWER

Share nothing. Then, in order: immutable data → `concurrent container` → `atomic` → `mutex`. Your own lock-free structure is the last resort, never the first move.

Dining philosophers: deadlock-free is not starvation-free

The puzzle is not about forks — it is about the gap between “nobody is stuck” and “everybody eats”.

All five grabbed their left fork



Grey: holds it. Red: waiting. The loop closes — circular wait.

Three fixes, three different promises

Fix	Deadlock	Starvation
Number the forks	gone	one may still never eat
A waiter hands out pairs	gone	the waiter is the new bottleneck
Chandy-Misra	gone	gone too

Every one of these promises assumes a fair scheduler underneath.

■ RULE OF THUMB

“It works” is not a guarantee. Name the one you actually need — no deadlock? no starvation? bounded wait? — and be ready to argue it.

Four thieves, one checklist

You were promised $N\times$. Here is who took it, and what you do about each one.

THIEF 1

The algorithm

Amdahl caps you at $1/(1 - p)$; contention bends the curve back down.

Counter-move: plot e as N grows. One number tells you nothing — the *trend* names the culprit.

THIEF 2

The hardware

Cores are fast; memory is not. Two threads on one cache line fight over it.

Counter-move: pad hot per-thread data to its own line. Check bandwidth first.

THIEF 3

The OS

Every kernel thread costs a stack and a slot; a switch costs microseconds.

Counter-move: never a thread per task. Pool sized to the cores; async for waiting.

THIEF 4

Your own locks

Whatever sits inside the critical section never scales.

Counter-move: shrink it. Move I/O, allocation and logging out; shard before you get clever.

■ RULE OF THUMB

Start by **sharing nothing**. When you must share, pick the simplest primitive that gives you the guarantee you need — then **measure**, because every thief on this slide is invisible until you do.